NAME:- Farooqui Shahzeb Ahmed

ROLL.NO:- SYA-29

ASSINGMENT:-08

AIM:-Implement and analyze various methods of studying correlation and regression including Karl Pearson's coefficient and simple linear regression.

THEORY

Correlation and regression are important statistical techniques used to analyze relationships between variables. Correlation measures the degree and direction of association between two variables, indicating whether they move together positively, negatively, or not at all. One of the most widely used methods is Karl Pearson's coefficient of correlation, which provides a numerical value ranging from -1 to +1 to express the strength and direction of a linear relationship. A value close to +1 indicates strong positive correlation, close to -1 indicates strong negative correlation, and near 0 indicates no correlation. Methods such as scatter diagrams offer a visual representation, while Karl Pearson's method gives a precise mathematical measure, making it highly useful in statistical analysis.

Regression analysis, on the other hand, is used to study the dependence of one variable on another and to predict values. In simple linear regression, the relationship between two variables is expressed in the form of an equation, typically Y=a+bX, where one variable is dependent and the other is independent. The regression coefficient indicates how much the dependent variable changes with a unit change in the independent variable. Unlike correlation, regression focuses on prediction and establishes a functional relationship. Both techniques are closely related, as correlation indicates the strength of the relationship, while regression provides a model for estimating and forecasting, making them essential tools in research, business, and data analysis.

Q1 **What is Correlation? How is it Different from Regression?**

ANS:-

correlation

Correlation is a statistical technique used to measure the strength and direction of the linear relationship between two quantitative variables. It answers the fundamental question: do these two variables tend to move together? When one variable increases, does the other increase as well (positive correlation), decrease (negative correlation), or show no consistent pattern (zero correlation)? The most widely used measure is Karl Pearson's correlation coefficient, denoted by r, which always lies between -1 and +1. A value of +1 indicates a perfect positive linear relationship, -1 indicates a perfect negative linear relationship, and 0 indicates no linear association between the variables whatsoever. The magnitude of r tells us about the strength: values of |r| above 0.7 are considered strong, values between 0.4 and 0.7 are moderate, and values below 0.4 are considered weak. It is important to remember that correlation measures only linear

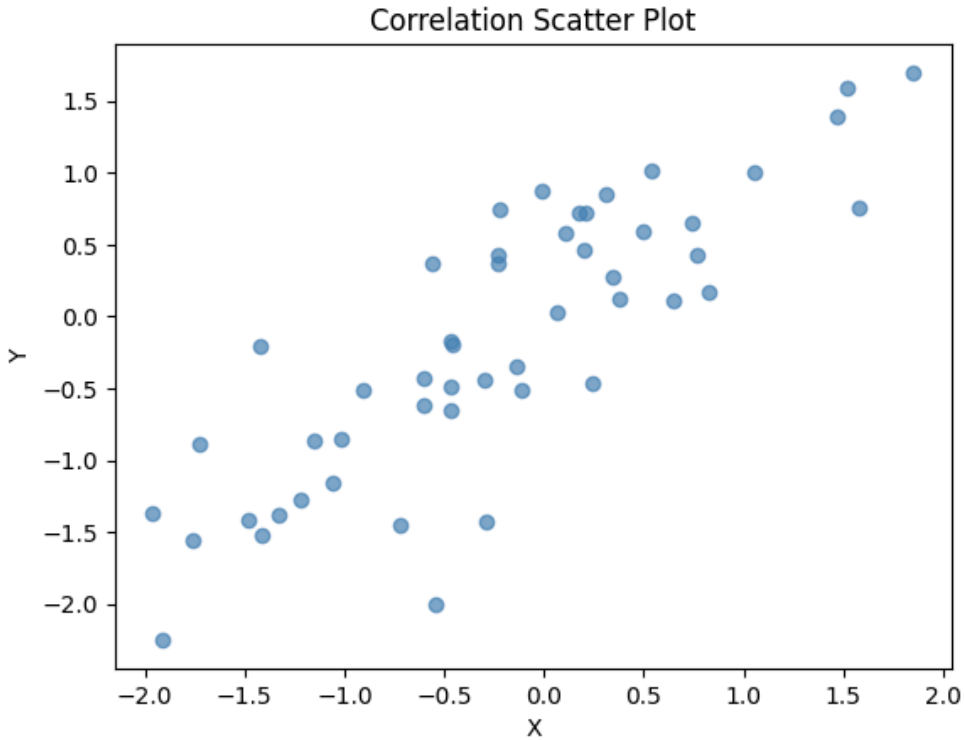
association — two variables can have a perfect non-linear relationship and still show r close to zero. Furthermore, correlation never implies causation; it simply describes how two variables co-vary.

How is Correlation Different from Regression?

While correlation and regression both deal with the relationship between two variables, they serve fundamentally different purposes. Correlation is purely descriptive and symmetric — swapping X and Y produces the exact same coefficient r. It simply quantifies how strongly two variables are related without specifying any direction of influence. Regression, on the other hand, is asymmetric and predictive. It explicitly treats one variable as the independent predictor (X) and the other as the dependent outcome (Y), and it constructs an equation of the form Y = b0 + b1*X that can be used to predict Y for any given value of X. Another key distinction is that regression implies a directional relationship and is used for forecasting, while correlation is used purely for exploration and understanding the degree of association. In practice, we first compute the correlation to check whether a meaningful relationship exists, and only then proceed to build a regression model for prediction. Correlation gives a single number; regression gives a full mathematical model.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
np.random.seed(42)
X = np.random.randn(50)
Y = 0.8 * X + np.random.randn(50) * 0.6
r, p = stats.pearsonr(X, Y)
print(f"Pearson r = {r:.4f}, p-value = {p:.4f}")
plt.scatter(X, Y, color='steelblue', alpha=0.7)
plt.title('Correlation Scatter Plot')
plt.xlabel('X'); plt.ylabel('Y')
plt.show()
```

```
Pearson r = 0.8392, p-value = 0.0000
```



Q2 **What is Simple Linear Regression? Explain with a Suitable Example**

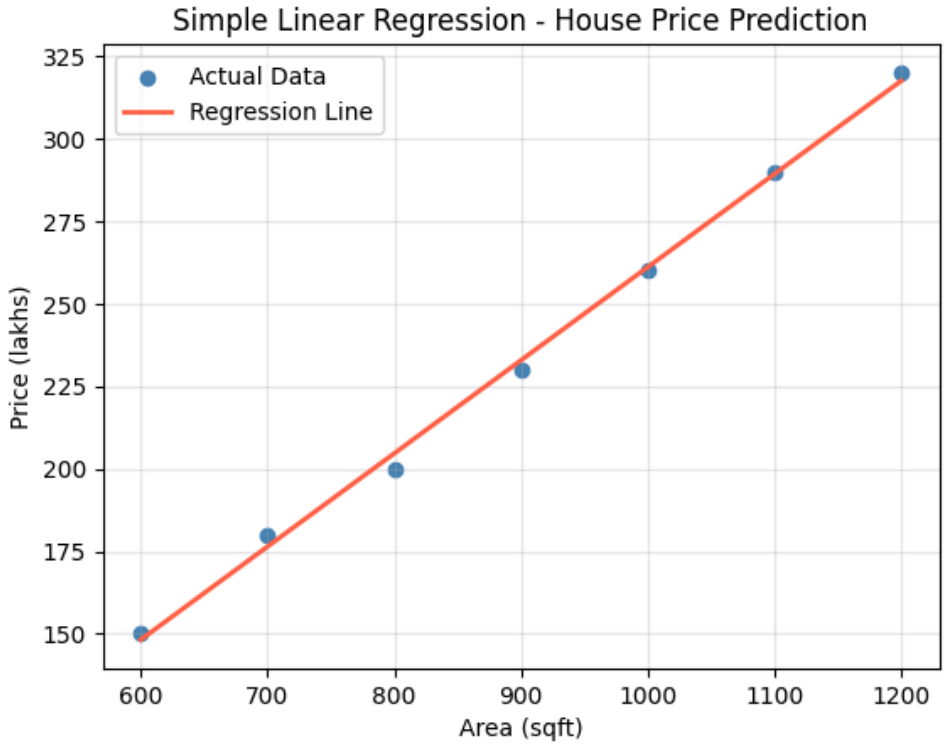
ANS:- Simple linear regression is a statistical method that models the relationship between a single independent variable (X) and a single dependent variable (Y) using a straight line. The goal is to find the best-fitting line through the data points such that the sum of the squared differences between the observed Y values and the predicted Y values (called residuals) is minimized. This method is known as Ordinary Least Squares (OLS). The resulting model allows us to predict the value of Y for any given value of X, making it one of the most foundational and widely used techniques in statistics and machine learning. The mathematical form of simple linear regression is expressed as Y = b0 + b1*X + e, where b0 is the intercept (the expected value of Y when X equals zero), b1 is the slope (the change in Y for every one-unit increase in X), and e is the error term that captures the unexplained variation in Y. The slope b1 is calculated as the ratio of the covariance of X and Y to the variance of X, while the intercept b0 is computed from the means of X and Y.

Example:-

Consider a real estate dataset where we want to predict house price (in lakhs) based on the area of the house in square feet. We have seven observations: areas of 600, 700, 800, 900, 1000, 1100, and 1200 sq ft correspond to prices of 150, 180, 200, 230, 260, 290, and 320 lakhs respectively. When we fit a simple linear regression model to this data, the algorithm finds the slope and intercept that best describe the linear trend. For instance, if the fitted equation turns out to be Price = -30 + 0.30 * Area, it means that for every additional square foot, the price increases by 0.30 lakhs, and the baseline value when area is zero is -30 lakhs (which is meaningless practically, but mathematically defines the line). Using this equation, we can predict that a house with 950 sq ft would cost approximately 255 lakhs.

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
import matplotlib.pyplot as plt
area = np.array([600,700,800,900,1000,1100,1200]).reshape(-1,1)
price = np.array([150,180,200,230,260,290,320])
model = LinearRegression()
model.fit(area, price)
print(f"Intercept (b0) : {model.intercept_:.2f}")
print(f"Slope (b1) : {model.coef_[0]:.4f}")
print(f"Predict 950sqft: {model.predict([[950]])[0]:.2f} lakhs")
print(f"R-squared : {r2_score(price, model.predict(area)):.4f}")
plt.scatter(area, price, color='steelblue', label='Actual Data')
plt.plot(area, model.predict(area), color='tomato', lw=2, label='Regression Line')
plt.xlabel('Area (sqft)'); plt.ylabel('Price (lakhs)')
plt.title('Simple Linear Regression - House Price Prediction')
plt.legend(); plt.grid(True, alpha=0.3); plt.show()
```

```
Intercept (b0) : -21.07
Slope (b1) : 0.2821
Predict 950sqft: 246.96 lakhs
R-squared : 0.9976
```



Q3 **How Do You Calculate Pearson Correlation in Python?**

ANS:- Karl Pearson's correlation coefficient (r) is computed by dividing the covariance of two variables by the product of their standard deviations. In practice, this equals the sum of the products of the z-scores of X and Y divided by (n-1). The formula captures both the direction and the magnitude of the linear relationship. A positive r means both variables tend to increase together; a negative r means one increases as the other decreases. The closer r is to +1 or -1, the stronger the linear relationship.

r = SUM[(Xi - X_mean)(Yi - Y_mean)] / SQRT[SUM(Xi-X_mean)^2 * SUM(Yi-Y_mean)^2]

```
import numpy as np
import pandas as pd
from scipy.stats import pearsonr

hours = np.array([2, 3, 4, 5, 6, 7, 8])
score = np.array([45, 55, 60, 70, 75, 85, 90])

r_numpy = np.corrcoef(hours, score)[0, 1]
print(f"Method 1 - NumPy : r = {r_numpy:.4f}")

r_scipy, p_val = pearsonr(hours, score)
print(f"Method 2 - SciPy : r = {r_scipy:.4f}, p-value = {p_val:.6f}")

df = pd.DataFrame({'Hours': hours, 'Score': score})
r_pandas = df.corr()['Hours']['Score']
print(f"Method 3 - Pandas : r = {r_pandas:.4f}")

if abs(r_numpy) >= 0.7:
    print("Interpretation: Strong linear correlation")
```

```
elif abs(r_numpy) >= 0.4:
    print("Interpretation: Moderate linear correlation")
else:
    print("Interpretation: Weak linear correlation")
```

```
Method 1 - NumPy : r = 0.9966
Method 2 - SciPy : r = 0.9966, p-value = 0.000001
Method 3 - Pandas : r = 0.9966
Interpretation: Strong linear correlation
```

Q4 **What is Multicollinearity? Does it Affect Simple Regression?**

ANS:- Multicollinearity is a phenomenon that occurs in multiple linear regression when two or more independent (predictor) variables are highly correlated with one another. When this happens, it becomes extremely difficult for the regression model to isolate the individual effect of each predictor on the dependent variable, because the predictors are essentially providing redundant information. For example, if we try to predict a student's grade using both 'hours studied' and 'pages read', and if students who study more hours also read more pages (making these two variables highly correlated), the model cannot determine whether the grade improvement is due to more hours, more pages, or some combination of both.

Does it Affect Simple Linear Regression?

No — multicollinearity does not and cannot affect simple linear regression. By definition, simple linear regression involves only one independent variable (X) predicting one dependent variable (Y). Since there is only a single predictor, there are no other predictors with which it can be correlated. Multicollinearity is exclusively a multiple regression problem that arises only when there are two or more predictors in the model. In simple regression, the slope coefficient b1 is always uniquely and stably determined, and there is no risk of inflation of standard errors due to inter-predictor correlation.

```
import numpy as np
import pandas as pd
from statsmodels.stats.outliers_influence import variance_inflation_factor
np.random.seed(42)
n = 100
X1 = np.random.randn(n)
X2 = X1 + np.random.randn(n) * 0.1 # Highly correlated with X1
X3 = np.random.randn(n) # Independent variable
df = pd.DataFrame({'X1': X1, 'X2': X2, 'X3': X3})
# Calculate VIF for each predictor
print("Variance Inflation Factors:")
for i, col in enumerate(df.columns):
    vif = variance_inflation_factor(df.values, i)
    flag = "HIGH - Multicollinearity!" if vif > 10 else "OK"
    print(f" VIF({col}) = {vif:.2f} --> {flag}")
# Correlation matrix
print()
print("Correlation Matrix:")
print(df.corr().round(3))
```

```
Variance Inflation Factors:
 VIF(X1) = 92.02 --> HIGH - Multicollinearity!
 VIF(X2) = 91.95 --> HIGH - Multicollinearity!
 VIF(X3) = 1.03 --> OK

Correlation Matrix:
```

```
          X1     X2     X3
    X1  1.000  0.994  0.191
    X2  0.994  1.000  0.189
    X3  0.191  0.189  1.000
```

Q5 **A Dataset Shows Strong Correlation but Poor Prediction Accuracy. Why?**

ANS:-

Understanding the Paradox

It is entirely possible — and in practice quite common — for a dataset to exhibit a strong Pearson correlation coefficient (r close to 1) yet produce poor prediction accuracy when a linear regression model is fitted. This apparent paradox arises because Pearson's r measures only the strength of the linear association between two variables, while prediction accuracy depends on how well the entire model captures the true underlying pattern in the data. Several distinct reasons can explain this discrepancy, and understanding them is critical to building reliable predictive models.

Reason 1: Non-linear Relationship

The most common cause is that the true relationship between X and Y is non-linear (for example, quadratic, exponential, or logarithmic), while both the correlation coefficient and the regression model assume linearity. Pearson's r can still show a moderate or even strong value for certain non-linear patterns, because the overall trend may be partially linear even if the detailed shape is curved. However, when a straight-line model is fitted to inherently curved data, the residuals are systematically large, resulting in a low R-squared and high mean squared error despite the seemingly decent correlation. The solution is to use polynomial regression or other non-linear models.

Reason 2: Influence of Outliers

Outliers and high-leverage data points can dramatically distort the Pearson correlation coefficient, pulling it closer to 1 or -1 without reflecting the true relationship in the majority of the data. A single extreme observation can inflate r significantly while simultaneously distorting the regression line away from the bulk of the data, causing terrible predictions for most observations. This is why it is always essential to visualise the data using scatter plots and to examine residual plots before trusting any statistical summary.

Reason 3: Overfitting on Small Samples

When the dataset is very small, a high correlation may simply be the result of random chance rather than a true underlying relationship. On a dataset with only five or ten observations, it is relatively easy to get r = 0.9 by luck, especially if the variables are jointly distributed in a way that happens to look linear for that particular small sample. When such a model is applied to new data, the prediction accuracy collapses because the model has essentially memorised the noise in the training set rather than learned a generalizable pattern.

Reason 4: Spurious Correlation Due to Confounding Variables

Sometimes two variables X and Y are both driven by a third hidden variable Z (a confounder), making them appear strongly correlated with each other even though neither directly causes the other. For example, ice cream sales and drowning rates are strongly correlated — not because ice cream causes drowning, but because both are driven by hot weather (Z). If we build a regression model using only X to predict Y without including Z, our predictions will be unreliable even though the correlation looks strong. This is the classic case of correlation without causation leading to poor

predictive models.

The Famous Anscombe's Quartet Anscombe's Quartet is the definitive illustration of this problem. It consists of four datasets that all have virtually identical statistical summaries — the same mean of X and Y, the same variance, the same Pearson r of approximately 0.816, and the same regression line equation. Yet when visualised, the four datasets look completely different: one is a clean linear relationship, one follows a perfect quadratic curve, one is perfectly linear except for a single outlier, and one has all X values at a single point except for one leverage point. A linear model fits only the first dataset well; for the other three, prediction accuracy is poor despite the identical r value.

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
from scipy.stats import pearsonr
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
np.random.seed(42)
X = np.linspace(0, 10, 100)
# True relationship is quadratic (non-linear)
Y_true = 2 * X**2 - 10*X + 5 + np.random.randn(100)*5
# Pearson r (only measures linear association)
r, _ = pearsonr(X, Y_true)
print(f"Pearson r = {r:.4f}")
# Fit LINEAR model to non-linear data
linear = LinearRegression()
linear.fit(X.reshape(-1,1), Y_true)
Y_linear = linear.predict(X.reshape(-1,1))
print(f"Linear R-squared = {r2_score(Y_true, Y_linear):.4f}")
print(f"Linear MSE = {mean_squared_error(Y_true, Y_linear):.2f}")
# Correct: Polynomial model
poly = make_pipeline(PolynomialFeatures(2), LinearRegression())
poly.fit(X.reshape(-1,1), Y_true)
Y_poly = poly.predict(X.reshape(-1,1))
print(f"Poly R-squared = {r2_score(Y_true, Y_poly):.4f}")
print(f"Poly MSE = {mean_squared_error(Y_true, Y_poly):.2f}")
```

```
Pearson r = 0.8765
Linear R-squared = 0.7682
Linear MSE = 260.07
Poly R-squared = 0.9819
Poly MSE = 20.29
```

CONCLUSION:-

This assignment explored the foundational concepts of correlation and regression analysis in statistics and data science. Correlation, measured by Karl Pearson's coefficient r, quantifies the strength and direction of

the linear relationship between two variables and ranges from -1 to +1. It is symmetric, descriptive, and does not imply causation. Regression, particularly simple linear regression, extends this by building a predictive equation Y = b0 + b1*X that allows us to forecast the value of one variable from another. The OLS method minimises residuals to find the best-fitting line, and the model quality is assessed using R-squared and MSE. Python offers three equivalent methods for computing Pearson's r — NumPy, SciPy, and Pandas — with SciPy additionally providing the p-value for significance testing. Multicollinearity, the dangerous inflation of standard errors caused by highly correlated predictors, affects only multiple regression models and has no impact on simple regression with a single predictor. It is detected using the Variance Inflation Factor (VIF), with values above 10 demanding corrective action. Finally, the paradox of strong correlation yet poor prediction accuracy arises from non-linearity, outliers, small samples, or spurious confounding relationships. Anscombe's Quartet remains the most powerful demonstration of why data visualisation and multi-metric model evaluation are non-negotiable in any rigorous statistical analysis. The overriding lesson of this assignment is that no single statistic — not even a high r — should ever be trusted in isolation. Always look at the full picture: visualise, test, validate, and interpret thoughtfully.

Close

Double-click (or enter) to edit